

МАТЕМАТИЧЕСКИЕ ОСНОВЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Каганов Ю.Т.

ЛЕКЦИЯ № 12

Метод опорных векторов (SVM)

**От геометрической интуиции к мощному
инструменту ИИ**

Содержание

1. История и основные идеи
2. Математические основы
3. Линейно разделимые данные
4. Ядерный трюк (Kernel Trick)
5. Случай неразделимых данных
6. Мультиклассовая классификация
7. Регрессия (SVR)
8. Практическая реализация
9. Примеры применения
10. Сравнение с другими методами
11. Преимущества и ограничения
12. Современные расширения

Историческая справка

Эволюция SVM

- **1992** – Владимир Вапник и Корнелия Кортес
- 1995** – Статья «Support-Vector Networks»
- 1990-е** – Развитие теории статистического обучения
- 2000-е** – Широкое внедрение в компьютерное зрение и биоинформатику
- 2010-е** – Интеграция с глубоким обучением
- *«SVM – мост между статистикой и машинным обучением»*

Геометрическая интуиция

- **Основная идея**
- **Задача:** Найти оптимальную разделяющую гиперплоскость
- Критерий: Максимальный зазор (margin) между классами
- Опорные векторы: Точки, определяющие положение гиперплоскости
- **Визуализация:**
- Класс 1: ○ ○ ○ | Класс 2: △ △ △
- ○ ○ | △ △
- ○ Hyperplane △
- Support | Support
- Vectors | Vectors

Математическая постановка

- **Линейно разделимый случай**
- **Дано:**
 - Обучающая выборка: $\{(x_1, y_1), \dots, (x_n, y_n)\}$
 - $y_i \in \{-1, +1\}$
 - Гиперплоскость: $w \cdot x + b = 0$
- **Цель:**
 - Максимизировать $\text{margin} = 2/\|w\|$
 - при условии: $y_i(w \cdot x_i + b) \geq 1, \forall i$

Решение задачи оптимизации

- **Двойственная задача (Lagrange)**

- **Исходная задача:**

- $\min (1/2 \|w\|^2)$
- при $y_i(w \cdot x_i + b) \geq 1$

- **Двойственная задача:**

- $\max \sum \alpha_i - 1/2 \sum \sum \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$
- при $\sum \alpha_i y_i = 0, \alpha_i \geq 0$

- **Решение:**

- $w = \sum \alpha_i y_i x_i$
- $b = y_i - w \cdot x_i$ для опорных векторов

Ядерный трюк (Kernel Trick)

- **Работа в пространствах высокой размерности**
- **Проблема:** Линейная неразделимость в исходном пространстве
- **Решение:**
 - 1. Отображаем данные в пространство высокой размерности
 - 2. Используем ядра для эффективных вычислений
 - 3. Никогда явно не вычисляем преобразование $\Phi(x)$
- **Формула:**
$$K(x_i, x_j) = \Phi(x_i) \cdot \Phi(x_j)$$

Популярные ядерные функции

Сравнение ядер

Тип ядра	Формула	Параметры	Применение
Линейное	$K(x,y) = x \cdot y$	-	Линейно разделимые данные
Полиномиальное	$K(x,y) = (\gamma x \cdot y + r)^d$	d, γ, r	Полиномиальные зависимости
RBF (Гауссово)	$K(x,y) = \exp(-\gamma \ x-y\ ^2)$	γ	Универсальное, плавные границы
Сигмоидальное	$K(x,y) = \tanh(\gamma x \cdot y + r)$	γ, r	Нейросетевые аналоги

SVM для неразделимых данных

- **Soft Margin классификация**
- **Вводим переменные ослабления ξ_i :**
- $\min \frac{1}{2}\|w\|^2 + C \sum \xi_i$
- при $y_i(w \cdot x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$
- **Параметр C:**
 - . Большое C: Малая ошибка, узкий зазор
 - . Малое C: Допустимы ошибки, широкий зазор

Подбор гиперпараметров

- **Кросс-валидация и поиск**
- **Ключевые параметры:**
 1. **C** – штраф за ошибки (Soft Margin)
 2. γ – ширина RBF ядра
 3. **degree** – степень полиномиального ядра
- **Методы поиска:**
 - . Grid Search
 - . Random Search
 - . Bayesian Optimization

Мультиклассовая классификация

- **Стратегии «один против всех»**
- **One-vs-Rest (OvR):**
 - Для каждого класса k :
 - Обучаем бинарный классификатор
 - Класс k vs все остальные
- **One-vs-One (OvO):**
 - Для каждой пары классов (i,j) :
 - Обучаем бинарный классификатор
 - Используем голосование

- Левые сингулярные векторы (U) – это собственные векторы матрицы $A A^T$.
- Правые сингулярные векторы (V) – это собственные векторы матрицы $A^T A$.
- Сингулярные числа (σ) – это квадратные корни из собственных значений этих матриц: $\sigma_i = \sqrt{\lambda_i}$.
- **Ключевой вывод:** SVD более универсален и устойчив, чем разложение по собственным векторам (работает с любыми, не только квадратными матрицами).

SVM для регрессии (SVR)

- **Support Vector Regression**
- **Идея:** Найти функцию, отклонение от которой не превышает ε
- **Задача оптимизации:**
- $\min \frac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$
- при $-\varepsilon - \xi_i^* \leq y_i - (w \cdot x_i + b) \leq \varepsilon + \xi_i$
- **ε -трубка:** Допустимая область ошибок

Практическая реализация

- Библиотеки и инструменты
- Python:
- python
- `from sklearn.svm import SVC, SVR`
- `from sklearn.model_selection import GridSearchCV`
- *# Классификация*
- `model = SVC(kernel='rbf', C=1.0, gamma='scale')`
- `model.fit(X_train, y_train)`
- *# Регрессия*
- `regressor = SVR(kernel='rbf', C=100, gamma=0.1)`

Пример: Классификация

- **Распознавание рукописных цифр (MNIST)**
- python
- `from sklearn import svm, datasets`
- `from sklearn.model_selection import train_test_split`
-
- *# Загрузка данных*
- `digits = datasets.load_digits()`
- `X_train, X_test, y_train, y_test = train_test_split(...)`
-
- *# Обучение SVM*
- `clf = svm.SVC(gamma=0.001, C=100.)`
- `clf.fit(X_train, y_train)`
-
- *# Точность: ~98-99%*

Пример: Анализ текста

- **Классификация тональности отзывов**

- python

- `from sklearn.feature_extraction.text import TfidfVectorizer`

- `from sklearn.svm import LinearSVC`

-

- *# Векторизация текста*

- `vectorizer = TfidfVectorizer()`

- `X = vectorizer.fit_transform(texts)`

-

- *# Лине́йный SVM для больших данных*

- `clf = LinearSVC()`

- `clf.fit(X, y)`

- **Преимущества:** Хорошая работа с разреженными данными

Сравнение с другими методами

SVM vs Neural Networks vs Random Forest

Критерий	SVM	Нейросети	Random Forest
Интерпретируемость	Средняя	Низкая	Высокая
Работа с малыми данными	Отлично	Плохо	Хорошо
Большие данные	Проблемы	Отлично	Хорошо
Нелинейность	Через ядра	Встроенная	Встроенная
Переобучение	Устойчив	Склонен	Устойчив

Преимущества SVM

- **Сильные стороны метода**
 1. **Эффективность в высоких размерностях**
 2. **Устойчивость к переобучению** (особенно с малыми данными)
 3. **Гибкость** через выбор ядра
 4. **Теоретическая обоснованность** (теория Вапника-Червоненкиса)
 5. **Глобальный оптимум** (выпуклая задача)

Ограничения и недостатки

- **Когда SVM не подходит**
 1. **Очень большие наборы данных** ($O(n^2)$ или $O(n^3)$)
 2. **Много шума в данных** (перекрывающиеся классы)
 3. **Мультиклассовые задачи** (требуют дополнительных стратегий)
 4. **Выбор ядра и параметров** (нет универсальных правил)
 5. **Чёрный ящик** (интерпретация сложна)

Масштабирование для больших данных

- **Методы ускорения**
 1. **Сэмплирование** (выборка представительных точек)
 2. **Пакетная обработка** (chunking)
 3. **Приближённые методы** (approximate SVM)
 4. **Распределённые вычисления**
 5. **GPU-ускорение**
- **Пример:** LibLinear для линейных SVM на больших данных

SVM в глубоком обучении

Интеграция с нейросетями

1. SVM как финальный слой в нейросетях
2. Ядерные методы в представлениях (Kernel on top of features)
3. Maximum Margin в обучении (Hinge loss)
 - python
 - *# SVM Loss в PyTorch*
 - `criterion = torch.nn.MultiMarginLoss(p=1, margin=1.0)`

Современные расширения

Актуальные разработки

1. **Structured SVM** — для структурного вывода
2. **Laplacian SVM** — полуавтоматическое обучение
3. **Budgeted SVM** — ограниченные ресурсы
4. **Online SVM** — потоковые данные
5. **Fuzzy SVM** — работа с нечёткими данными

•

Применение в биоинформатике

Анализ биологических данных

- . Классификация белков**
- . Анализ экспрессии генов**
- . Распознавание ДНК-последовательностей**
- . Медицинская диагностика**
- Пример: Обнаружение рака по микроматричным данным**

Применение в компьютерном зрении

- **Ключевые задачи**

1. **Распознавание объектов**

2. **Детектирование лиц**

3. **Классификация сцен**

4. **Слежение за объектами**

- **Histogram of Oriented Gradients (HOG) + SVM –
классический пайплайн**

Будущее SVM

- **Перспективы развития**

1. **Интеграция с deep learning**

2. **Автоматический выбор ядер**

3. **Квантовые SVM**

4. **Интерпретируемые SVM**

5. **SVM для графовых данных**

- *«SVM остаётся важным инструментом в арсенале data scientist»*

Итоги и практические советы

- **Когда использовать SVM:**
 - Мало обучающих данных
 - Высокая размерность признаков
 - Требуется устойчивость к переобучению
 - Чёткая граница между классами
- **Практические рекомендации:**
 1. Начинайте с линейного ядра
 2. Используйте кросс-валидацию для выбора C и γ
 3. Нормализуйте данные
 4. Для больших данных используйте LinearSVC
 5. Экспериментируйте с разными ядрами